Lab 2 – Deep Learning
Implementing a Neural Network and Backpropagation

Objective

The goal of this lab is to implement, **without using any deep learning library** (no `scikit-learn`, `TensorFlow`, `PyTorch`, etc.), a multilayer neural network (multilayer perceptron) together with the backpropagation algorithm used to train it. The lab is organized as a series of progressive exercises that will lead you to:

- build and represent a neural network in Python (lists of dictionaries);
- compute the forward propagation;
- compute the backpropagation of the error;
- update the network's weights;
- train the network on a toy dataset, then on a real dataset (*Seeds Dataset*);
- generalize your implementation to an arbitrary number of hidden layers and to different activation functions / loss functions.

You should only use the `random`, `math` and `csv` modules from the Python standard library.

1 Network initialization

A neural network will be represented as a list of layers, each layer itself being a list of neurons. Each neuron is a dictionary containing (at least) a key `'weights'` associated with the list of its weights (including the bias, placed last).

Question 1. Write a function

```
initialize_network(n_inputs, n_hidden, n_outputs)
```

that builds a network with a single hidden layer of `n_hidden` neurons, an input layer of size `n_inputs` and an output layer of size `n_outputs`. Weights should be initialized randomly (values drawn uniformly in $[0, 1[$, rounded to one decimal place).

Question 2. Write a function `write(network)` that prints, layer by layer, the contents of the network (useful for debugging).

Question 3. Test your function on a network with 2 inputs, 3 hidden neurons and 4 outputs.

2 Checking the initialization

Question 4. Fixing the random seed (`seed(1)`), initialize a network with 3 inputs, 2 hidden neurons and 2 outputs, then print its contents. Check that the execution is reproducible.

3 Neuron activation

Question 5. Write a function

`activate(weights, inputs)`

that computes the activation (pre-activation, i.e. the linear combination of the inputs and the weights, plus the bias) of a neuron:

$$a = \sum_i w_i \, x_i + w_{\text{bias}}$$

Question 6. Test this function on `weights = [3.0, 5.0, 2]` and `inputs = [-1, 1]`.

4 Transfer function

Question 7. Write a function `transfer(activation)` that applies the logistic (sigmoid) function to the pre-activation computed above:

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

5 Forward propagation

Question 8. Write a function

`forward_propagate(network, row)`

that propagates an example `row` through all the layers of the network, using the `activate` and `transfer` functions defined above. The output of each neuron (after applying the transfer function) should be stored in the neuron itself (key `'output'`), so that it can be reused during backpropagation.

Question 9. Test forward propagation on a network initialized with `initialize_network(2, 1, 2)` and the example `row = [1, 0]`. Print the network before and after propagation, as well as the resulting output.

6 Derivative of the transfer function

Question 10. Write a function `transfer_derivative(output)` that computes the derivative of the sigmoid *as a function of its output* $y = \sigma(a)$:

$$\sigma'(a) = y \, (1 - y)$$

7 Backpropagation of the error

Recall that the error δ of an output-layer neuron is given by:

$$\delta_j = (\text{expected}_j - \text{output}_j) \times \text{transfer_derivative}(\text{output}_j)$$

and that, for a neuron j in a hidden layer, the error is propagated backward from the next layer according to:

$$\delta_j = \Big(\sum_k w_{kj} \, \delta_k \Big) \times \text{transfer_derivative}(\text{output}_j)$$

where the sum runs over all neurons k of the next layer connected to neuron j.

Question 11. Write a function

`backward_propagate_error(network, expected)`

that computes, for every neuron in the network (starting from the output layer and moving back toward the input layer), the value of δ and stores it in the neuron (key `'delta'`).

Question 12. Test this function on the network obtained in Question 9, with `expected = [0, 1]`. Print the network before and after the call.

8 Weight update

Once the δ values have been computed, the weights are updated using the gradient rule:

$$w_i \leftarrow w_i + \eta\, \delta\, x_i, \qquad w_{\text{bias}} \leftarrow w_{\text{bias}} + \eta\, \delta$$

where η is the learning rate and x_i is the input received by the neuron (the output of the previous layer, or the network input for the first layer).

Question 13. Write a function

`update_weights(network, row, l_rate)`

that updates the weights of the whole network from the δ values computed by `backward_propagate_error`.

9 Training the network

Question 14. Write a function

`train_network(network, train, l_rate, n_epoch, n_outputs)`

that trains the network for `n_epoch` epochs on the dataset `train` (each row of `train` being an example whose last column is the expected class, encoded as an integer). For each example, you will perform:

1. a forward propagation;
2. the construction of the expected *one-hot* vector from the class label;
3. the computation of the squared error (sum of squared differences) and its accumulation over the epoch;
4. a backpropagation of the error;
5. a weight update.

At each epoch, print the epoch number, the learning rate, and the sum of squared errors.

Question 15. Test the training on the following small two-class toy dataset, with a network of 2 hidden neurons, a learning rate of 0.5, and 40 epochs:

```
dataset = [[2.7810836,2.550537003,0],
    [1.465489372,2.362125076,0],
    [3.396561688,4.400293529,0],
    [1.38807019,1.850220317,0],
    [3.06407232,3.005305973,0],
    [7.627531214,2.759262235,1],
    [5.332441248,2.088626775,1],
    [6.922596716,1.77106367,1],
    [8.675418651,-0.242068655,1],
    [7.673756466,3.508563011,1]]
```

10 Prediction

Question 16. Write a function `predict(network, row)` that returns the class predicted by the network for a given example (index of the maximum output after forward propagation).

Question 17. Use this function to print, for each example in the dataset above, the expected class and the class predicted by the trained network.

11 Bonus question – generalization

The following questions are optional and extend the implementation above.

Question 18. Modify your `activate`, `forward_propagate` and `transfer_derivative` functions to allow the choice, **layer by layer**, of an activation function among: `linear`, `relu`, `sigmoid` and `tanh`, as well as a `softmax` activation for the output layer (useful for multi-class classification). You may add an optional `activations` parameter (a list of strings, one per layer) to the relevant functions.

Question 19. Modify `backward_propagate_error` and `train_network` to allow the choice of a loss function (`loss`) among mean squared error (`'mse'`) and cross-entropy (`'cross_entropy'`). Recall that when the output layer uses `softmax` (or `sigmoid`) combined with a cross-entropy loss, the expression of the output δ simplifies to:

$$\delta_j = \hat{y}_j - y_j.$$

Question 20. Using your generalized implementation, compare different configurations (activation functions / loss function) on:

1. a binary classification dataset;
2. a 3-class classification dataset;
3. a regression dataset (for example $y \approx 3x_1 - 2x_2 + 0.5$).

For each configuration, display the evolution of the error during training as well as the final performance (accuracy for classification problems, predictions compared to true values for regression).

12 Application to a real dataset: the Seeds Dataset

We now want to apply your implementation to a real dataset, the *Seeds Dataset*, which contains measurements taken on wheat kernels belonging to three different varieties.

Question 21. Load the file `seeds_dataset.csv` (on Google Colab, you can use `google.colab.files.uploa` or mount your Google Drive). Write a function `load_csv(filename)` to read the file, then convert each row into a list of floating-point numbers (the last column, corresponding to the class, should be an integer starting at 0).

Question 22. Automatically determine, from the loaded dataset, the number of inputs and the number of classes (outputs) for the network.

Question 23. Initialize a suitable network (for example, 10 hidden neurons), train it on the *Seeds Dataset*, and print, for each example, the expected class and the predicted class.

13 Bonus question – network with several hidden layers

Question 24. Generalize the `initialize_network` function (you may call it `initialize_network_multiple_h` so that it accepts, instead of a single integer `n_hidden`, a **list** giving the number of neurons in each hidden layer (for example `[3, 5, 4]` for a network with three hidden layers). Check that,

when this list contains only one element, your function behaves like the function from Question 1.

Question 25. Check that the `forward_propagate` and `backward_propagate_error` functions work without modification on a network with several hidden layers (justify why).